

Module 7 – Modules and Packages

Overview

As Python programs grow in size, writing all code in a single file becomes difficult to manage and maintain. This module introduces modules and packages, which are Python's way of organizing code into reusable and structured units. Students will learn how to create and use built-in modules, user-defined modules, and third-party packages, understand different import techniques, and organize real-world projects using packages.

7.1. What Is a Module?

A module is a single Python file with a .py extension that contains variables, functions, classes, or executable code. Modules allow developers to logically separate functionality and reuse code across multiple programs.

```
# utils.py
def add(a, b):
    return a + b

# main.py
import utils
print(utils.add(2, 3))
```

7.2. Import Statements

Python provides multiple ways to import modules depending on how much functionality is required. You can import the entire module, import specific names, or use aliases to simplify access and avoid naming conflicts.

```
import math as m
from datetime import datetime as dt
```

7.3. The Dunder (__) Convention

Names that begin and end with double underscores are called dunder names. They have special meanings in Python. The most commonly used is `__name__`, which helps distinguish whether a file is being executed directly or imported as a module.

```
if __name__ == "__main__":
    print("Running as a script")
```

7.4. Reloading a Module in Python

- What is Module Reloading?

During development, a Python module's code may change frequently. Normally, once a module is imported, Python does not automatically reload it, even if the source file is modified.

To apply changes without restarting the Python interpreter, Python provides the `importlib.reload()` function.

- **Why Module Reloading is Needed?**
 1. Faster development and testing
 2. Useful in interactive mode, IDLE, Jupyter Notebook, and REPL
 3. Avoid restarting the interpreter after every code change

importlib Module

The `importlib` module provides tools related to the import system. The `reload()` function is part of this module.

Example

Step 1: Create a module `mymodule.py`

```
def greet():  
    print("Hello from version 1")
```

Step 2: Import and use it

```
import mymodule  
mymodule.greet()
```

Output:

```
Hello from version 1
```

Step 3: Modify `mymodule.py`

```
def greet():  
    print("Hello from version 2")
```

Step 4: Reload the module

```
import importlib  
importlib.reload(mymodule)  
mymodule.greet()
```

Output:

```
Hello from version 2
```

7.5. Built-in Modules

Python includes a rich standard library of built-in modules that provide ready-made functionality such as mathematical operations, file handling, operating system interaction, and system-level access.

```
import sys  
print(sys.version)
```

7.6. User-Defined Modules

User-defined modules are custom Python files created by developers to organize project-specific logic. They improve readability and allow teams to collaborate by separating responsibilities.

7.7. Third-Party Modules and pip

Third-party modules are developed by the Python community and distributed through the Python Package Index (PyPI). They can be installed using the pip tool and imported like any other module.

```
pip install requests

import requests
response = requests.get('https://api.example.com')
```

7.8. sys.path

Python searches for modules in a list of directories defined in sys.path. Custom directories can be added at runtime if modules are stored outside standard locations.

```
import sys
sys.path.append(r'C:/my/custom/modules')
```

7.9. Packages and Sub-Packages

A package is a directory that groups related modules together. It must contain an `__init__.py` file to be recognized as a package. Packages help structure large applications into logical components.

```
myproject/
├── __init__.py
├── core.py
├── utils.py
└── subpkg/
    ├── __init__.py
    └── helpers.py
```

7.10. Namespaces

Each module in Python has its own namespace, which maps names to objects. Namespaces prevent naming conflicts by keeping identifiers isolated within their modules.

7.11. Modular Programming and Code Reuse

Modular programming promotes separation of concerns, code reuse, and easier maintenance. Well-structured modules and packages make applications scalable and easier to test.

Sample Program – Using Modules and Packages

```
# mypackage/math_ops.py
def square(x):
```

```
    return x * x

def cube(x):
    return x ** 3

# main.py
from mypackage.math_ops import square, cube
print(square(4))
print(cube(2))
```

